
Marketplace Revenue Leakage & Gap Analysis using SQL

Presented by

D. Jaya Kumar

GUVI | SQL Project Presentation

Dataset Overview

ORIGINAL DATASET TABLES

<code>products_v2</code>	Product details and cost
<code>orders_v2</code>	Order transactions
<code>discounts_v2</code>	Discount details
<code>returns_v2</code>	Return flag and reasons
<code>logistics_cost_v2</code>	Shipping and reverse shipping
<code>payment_fees_v2</code>	Payment method fee %

CLEAN-TABLE ROW COUNTS

table_name	row_count
products_clean	1431
orders_clean	17441
discounts_clean	7140
returns_clean	3148
logistics_clean	10242
payment_fees_clean	7

Task 1 — Revenue vs Profit Reality

PROBLEM STATEMENT

Calculate total sales revenue, total product cost and total basic profit across all orders.

SQL CONCEPTS USED

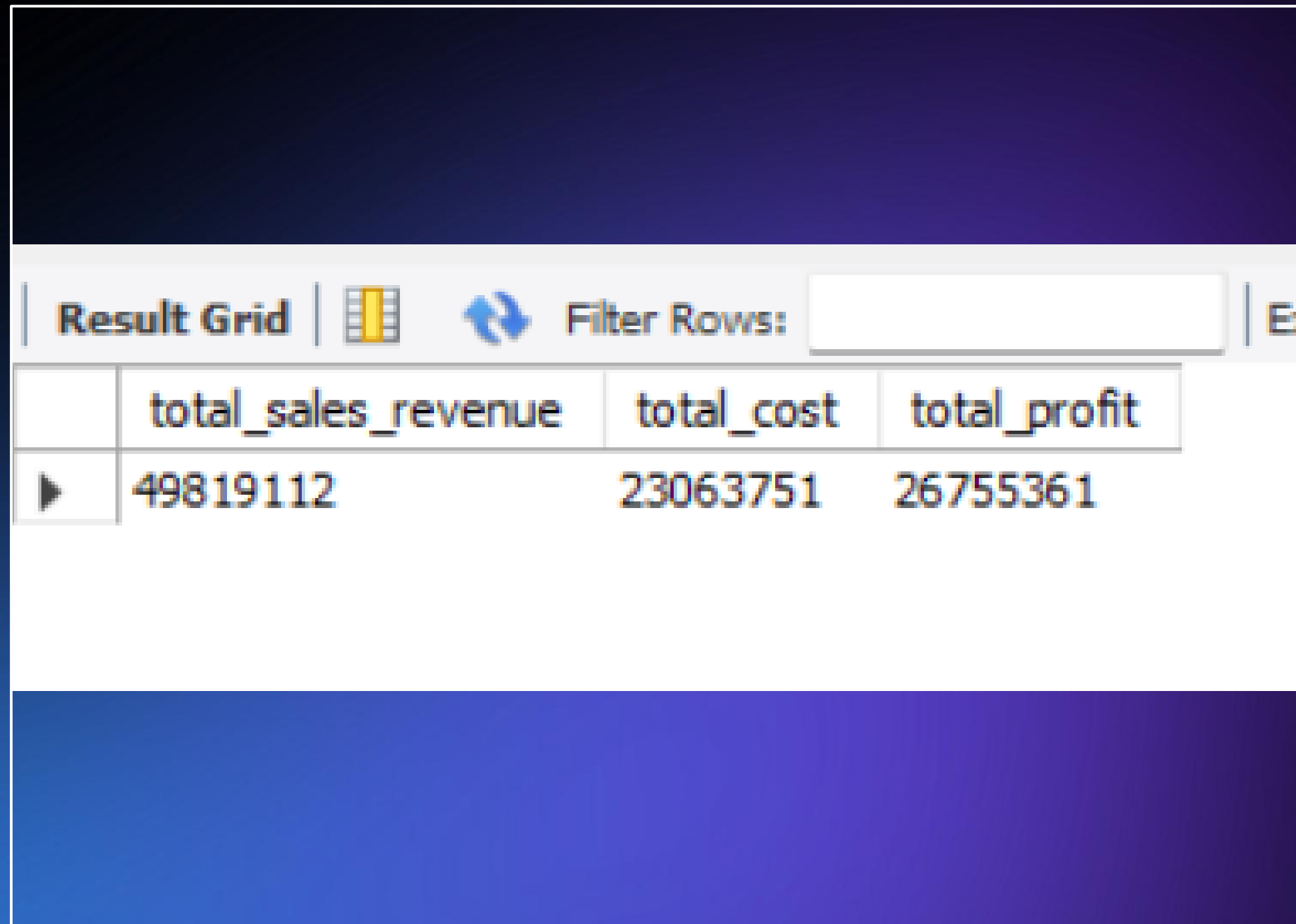
- SELECT
- SUM
- INNER JOIN
- Arithmetic Operations

QUERY USED

```
3 • SELECT
4     SUM(o.quantity * o.selling_price) AS total_sales_revenue,
5     SUM(o.quantity * p.cost_price) AS total_cost,
6     SUM(
7         (o.quantity * o.selling_price)
8         - (o.quantity * p.cost_price)
9     ) AS total_profit
10 FROM orders_clean AS o
11 INNER JOIN products_clean AS p
12     ON o.product_id = p.product_id;
```

Task 1 — Output & Finding

OUTPUT SCREENSHOT



The screenshot shows a SQL query result grid. The grid has a header row with columns: total_sales_revenue, total_cost, and total_profit. Below the header is a single data row with values: 49819112, 23063751, and 26755361. The grid is part of a larger interface with a 'Result Grid' tab and a 'Filter Rows' input field.

	total_sales_revenue	total_cost	total_profit
▶	49819112	23063751	26755361

RESULT

Sales ₹4,98,19,112; product cost ₹2,30,63,751; basic profit ₹2,67,55,361.

INSIGHT

This is the starting point before checking other leakage factors.

TAKEAWAY

Use this as the starting point for later leakage analysis.

Task 2 — Category-wise Sales & Profit

PROBLEM STATEMENT

Calculate total sales, cost and profit for each product category.

SQL CONCEPTS USED

- SELECT
- SUM
- INNER JOIN
- GROUP BY
- ORDER BY
- Arithmetic Operations

QUERY USED

```
SELECT
    p.category,

    SUM(o.quantity * o.selling_price) AS total_sales,

    SUM(o.quantity * p.cost_price) AS total_cost,

    SUM(
        (o.quantity * o.selling_price)
        - (o.quantity * p.cost_price)
    ) AS total_profit

FROM orders_clean AS o

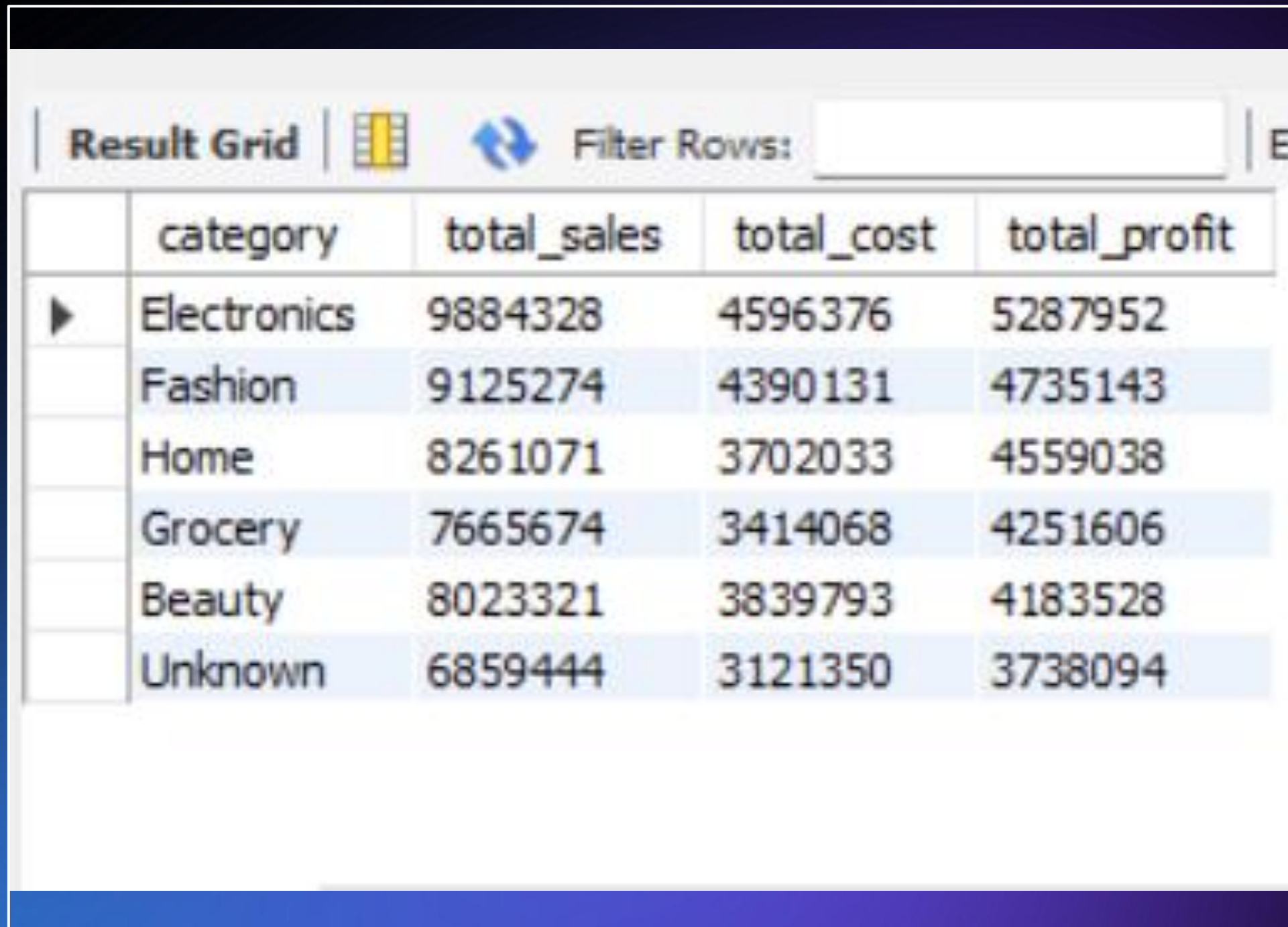
INNER JOIN products_clean AS p
    ON o.product_id = p.product_id

GROUP BY p.category

ORDER BY total_profit DESC;
```

Task 2 — Output & Finding

OUTPUT SCREENSHOT



The screenshot shows a database interface with a 'Result Grid' tab. It displays a table with 5 columns: category, total_sales, total_cost, and total_profit. The data is as follows:

	category	total_sales	total_cost	total_profit
▶	Electronics	9884328	4596376	5287952
	Fashion	9125274	4390131	4735143
	Home	8261071	3702033	4559038
	Grocery	7665674	3414068	4251606
	Beauty	8023321	3839793	4183528
	Unknown	6859444	3121350	3738094

RESULT

Electronics had the highest basic profit: ₹52,87,952.

INSIGHT

Category profitability differs, so category-level review is useful.

TAKEAWAY

Category has to be added so the unknown can be used for documentation.

Task 3 — Loss-Making Products

PROBLEM STATEMENT

Identify products whose total profit across orders is negative.

SQL CONCEPTS USED

- SELECT
- SUM
- INNER JOIN
- GROUP BY
- HAVING
- ORDER BY
- Arithmetic Operations

QUERY USED

```
• SELECT
    o.product_id,
    SUM(o.quantity * o.selling_price) AS total_sales,
    SUM(o.quantity * p.cost_price) AS total_cost,
    SUM(
        (o.quantity * o.selling_price)
        - (o.quantity * p.cost_price)
    ) AS total_profit
FROM orders_clean o
JOIN products_clean p
    ON o.product_id = p.product_id
GROUP BY o.product_id
HAVING total_profit < 0
ORDER BY total_profit ASC;
```

Task 3 — Output & Finding

OUTPUT SCREENSHOT

Result Grid				
Filter Rows:				
	product_id	total_sales	total_cost	total_profit
▶	521	34014	42679	-8665
	1358	25349	31540	-6191
	1661	41748	47488	-5740
	735	7064	12750	-5686
	964	25839	31324	-5485
	645	22342	27156	-4814
	382	11794	16188	-4394
	652	42765	47088	-4323
	1134	10010	14184	-4174
	351	14655	18600	-3945

RESULT

Product 521 had the highest basic loss: -₹8,665.

INSIGHT

Sales and profit can vary. More sales ≠ More Profit

ACTION

Review loss-making products before selling it.

Task 4 — Discount Usage Overview

PROBLEM STATEMENT

Find how many orders received discounts and calculate the total discount amount.

SQL CONCEPTS USED

- SELECT
- COUNT(DISTINCT)
- SUM

QUERY USED

```
SELECT  
    COUNT(DISTINCT order_id) AS discounted_orders,  
    SUM(discount_amount) AS total_discount_amount  
FROM discounts_clean;
```

Task 4 — Output & Finding

OUTPUT SCREENSHOT

	discounted_orders	total_discount_amount
▶	5873	4583197

RESULT

5,873 orders received discounts; total discounts ₹45,83,197.

INSIGHT

As this point of analysis discount seems high.

ACTION

Tracking of discount is highly recommendable.

Task 5 — Payment Method Popularity

PROBLEM STATEMENT

Compare order count and sales value by payment method.

SQL CONCEPTS USED

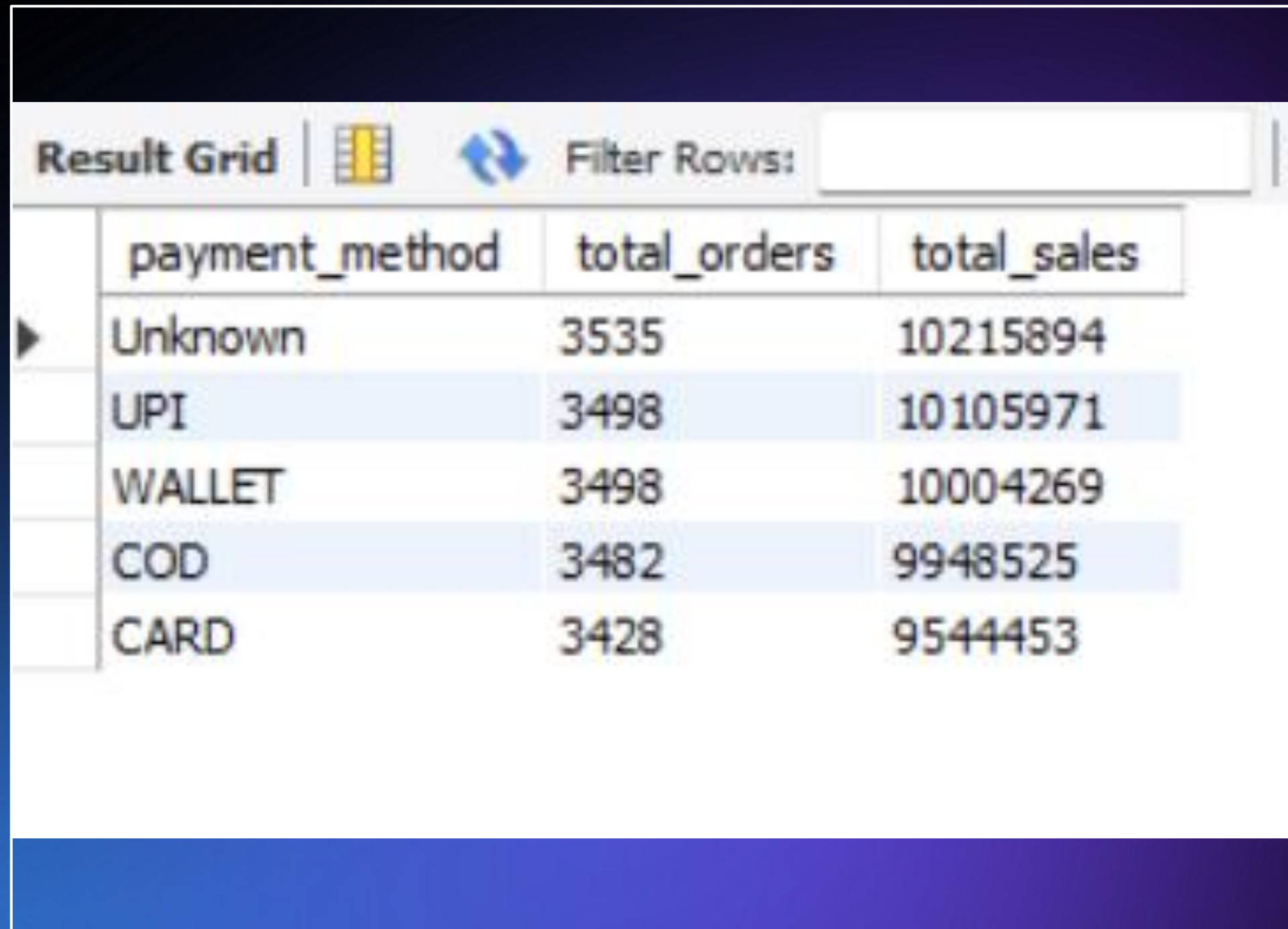
- SELECT
- COUNT
- SUM
- GROUP BY
- ORDER BY

QUERY USED

```
SELECT  
    payment_method,  
    COUNT(order_id) AS total_orders,  
    SUM(quantity * selling_price) AS total_sales  
FROM orders_clean  
GROUP BY payment_method  
ORDER BY total_orders DESC;
```

Task 5 — Output & Finding

OUTPUT SCREENSHOT



The screenshot shows a 'Result Grid' interface with a table of payment methods. The table has three columns: 'payment_method', 'total_orders', and 'total_sales'. The rows are: Unknown (3535 orders, 10215894 sales), UPI (3498 orders, 10105971 sales), WALLET (3498 orders, 10004269 sales), COD (3482 orders, 9948525 sales), and CARD (3428 orders, 9544453 sales). The 'Unknown' row is highlighted in yellow, and the other rows are highlighted in light blue. The interface includes a 'Filter Rows' button and a 'Result Grid' label.

	payment_method	total_orders	total_sales
▶	Unknown	3535	10215894
	UPI	3498	10105971
	WALLET	3498	10004269
	COD	3482	9948525
	CARD	3428	9544453

RESULT

Unknown had 3,535 orders. UPI and Wallet each had 3,498 orders.

INSIGHT

Among valid methods, UPI and Wallet were highly used. Unknown shows a data-quality issue.

ACTION

Improve payment-method data quality before deeper decisions.

Task 6 — Discount vs Profit Gap

PROBLEM STATEMENT

Compare average profit per order between discounted and non-discounted orders.

SQL CONCEPTS USED

- SELECT
- CASE
- COUNT
- ROUND
- AVG
- INNER JOIN
- LEFT JOIN
- Subquery
- GROUP BY

QUERY USED

```
SELECT
  CASE
    WHEN d.total_discount > 0 THEN 'Discounted'
    ELSE 'Non-Discounted'
  END AS discount_status,

  COUNT(o.order_id) AS total_orders,

  ROUND(
    AVG(
      CASE
        WHEN d.total_discount IS NULL THEN
          (o.quantity * o.selling_price)
          - (o.quantity * p.cost_price)
        ELSE
          (o.quantity * o.selling_price)
          - (o.quantity * p.cost_price)
          - d.total_discount
        END
      ), 2

FROM orders_clean o

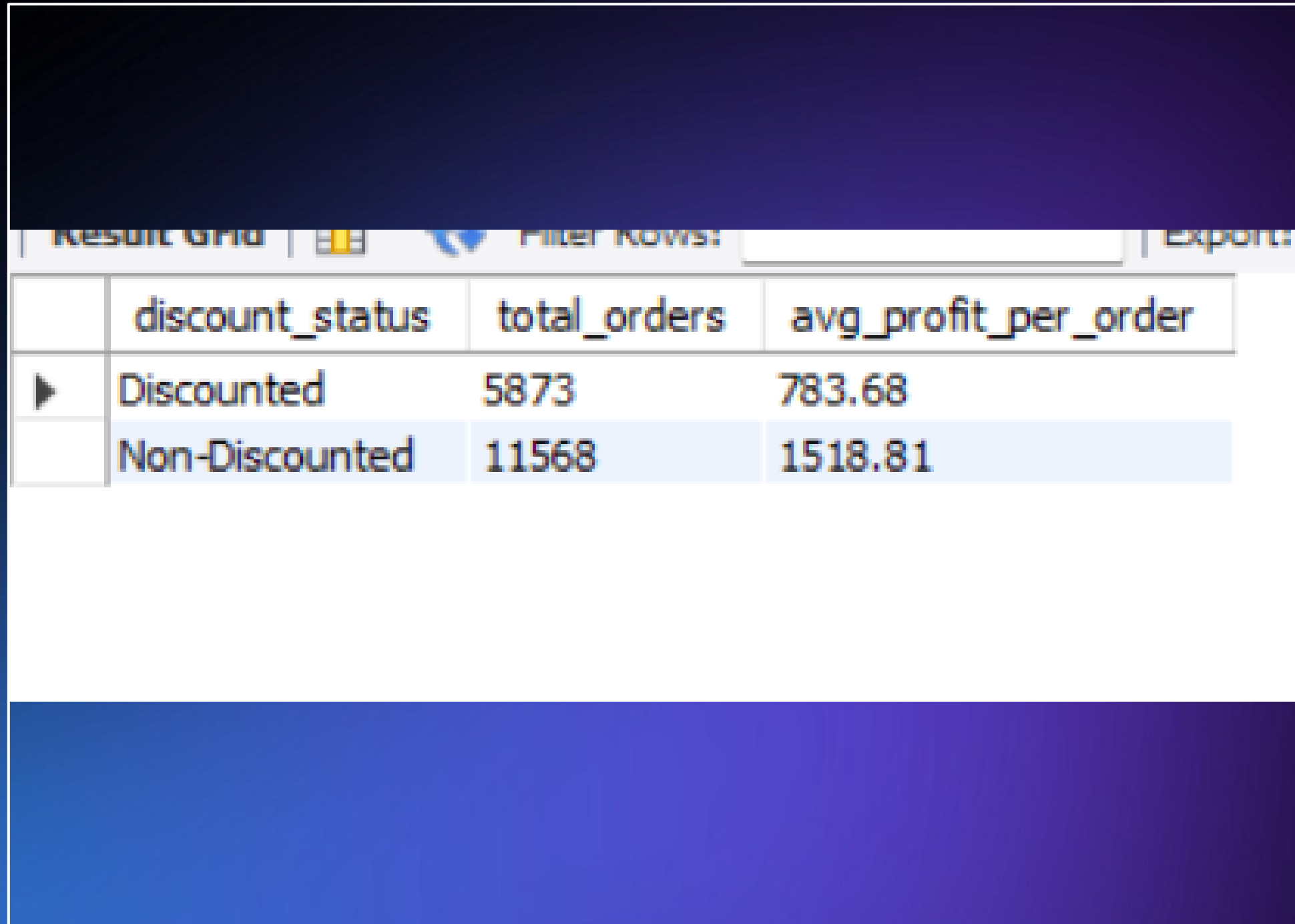
JOIN products_clean p
  ON o.product_id = p.product_id

LEFT JOIN
(
  SELECT
    order_id,
    SUM(discount_amount) AS total_discount
  FROM discounts_clean
  GROUP BY order_id
) d
  ON o.order_id = d.order_id

GROUP BY
  CASE
    WHEN d.total_discount > 0 THEN 'Discounted'
    ELSE 'Non-Discounted'
  END;
```

Task 6 — Output & Finding

OUTPUT SCREENSHOT



The screenshot shows a database query result grid. The top part of the grid is obscured by a dark blue rectangle. Below this, the grid header shows columns: discount_status, total_orders, and avg_profit_per_order. The first row shows 'Discounted' with 5873 total orders and an average profit of 783.68. The second row shows 'Non-Discounted' with 11568 total orders and an average profit of 1518.81. The 'Non-Discounted' row is highlighted in light blue. Below the grid, there is a large blue rectangle.

	discount_status	total_orders	avg_profit_per_order
▶	Discounted	5873	783.68
	Non-Discounted	11568	1518.81

RESULT

Discounted avg profit ₹783.68; non-discounted avg profit ₹1,518.81.

INSIGHT

Discounted orders produced much lower average profit per order.

ACTION

Review discounts and implement if based on needs only.

Task 7 — Return Impact on Revenue

PROBLEM STATEMENT

Calculate revenue and basic profit affected by confirmed returned orders.

SQL CONCEPTS USED

- SELECT
- ROUND
- SUM
- INNER JOIN
- Subquery
- DISTINCT
- WHERE
- Arithmetic Operations

QUERY USED

```
SELECT
    ROUND(
        SUM(o.quantity * o.selling_price), 2
    ) AS total_revenue_lost,

    ROUND(
        SUM(
            (o.quantity * o.selling_price)
            - (o.quantity * p.cost_price)
        ), 2
    ) AS total_profit_lost

FROM orders_clean o

JOIN products_clean p
    ON o.product_id = p.product_id

JOIN
    (
        SELECT DISTINCT order_id
        FROM returns_clean
        WHERE return_flag = 'Y'
    ) r
    ON o.order_id = r.order_id;
```

Task 7 — Output & Finding

OUTPUT SCREENSHOT

Result Grid		
Filter Rows:		
	total_revenue_lost	total_profit_lost
▶	2921684	1591717

RESULT

Revenue affected ₹29,21,684; basic profit affected ₹15,91,717.

INSIGHT

Returns have a big financial impact on the business.

ACTION

Reduce preventable returns and review return reasons.

Task 8 — Return Reason Analysis

PROBLEM STATEMENT

Count confirmed returns by reason and measure revenue associated with each reason.

SQL CONCEPTS USED

- SELECT
- COUNT
- ROUND
- SUM
- Subquery
- DISTINCT
- WHERE
- INNER JOIN
- GROUP BY
- ORDER BY

QUERY USED

```
SELECT
    r.return_reason,
    COUNT(*) AS total_returns,
    ROUND(
        SUM(o.quantity * o.selling_price),
        2
    ) AS revenue_lost
FROM
    (
        SELECT DISTINCT
            order_id,
            return_reason
        FROM returns_clean
        WHERE return_flag = 'Y'
    ) r
JOIN orders_clean o
    ON r.order_id = o.order_id
GROUP BY r.return_reason
ORDER BY revenue_lost DESC;
```

Task 8 — Output & Finding

OUTPUT SCREENSHOT

Result Grid   Filter Rows: Export: 			
	return_reason	total_returns	revenue_lost
▶	Unknown	195	579157
	Damaged	166	500686
	Wrong Item	175	483870
	Customer Changed Mind	181	480220
	Quality Issue	174	477882
	Late Delivery	159	463818

RESULT

Unknown was highest at ₹5,79,157. Damaged was next at ₹5,00,686.

INSIGHT

Return reason capture needs improvement, and damage prevention should be reviewed.

ACTION

Improve reason capture and focus on damage prevention.

Task 9 — Logistics Cost Burden

PROBLEM STATEMENT

Identify orders where logistics cost is more than 20% of order value.

SQL CONCEPTS USED

- SELECT
- ROUND
- CONCAT
- SUM()
- CASE
- Subquery
- INNER JOIN
- WHERE
- GROUP BY
- ORDER BY
- Percentage Calculation

QUERY USED

```

SELECT
  o.order_id,
  ROUND(o.quantity * o.selling_price, 2) AS order_value,
  ROUND(l.total_logistics_cost, 2) AS total_logistics_cost,
  CONCAT(
    ROUND(
      (l.total_logistics_cost / (o.quantity * o.selling_price)) * 100,
      2
    ),
    '%'
  ) AS logistics_burden_percent
FROM orders_clean o

JOIN
(
  SELECT
    order_id,
    SUM(
      CASE
        WHEN shipping_cost IS NULL THEN 0
        ELSE shipping_cost
      END
    )
    +
    CASE
      WHEN reverse_shipping_cost IS NULL THEN 0
      ELSE reverse_shipping_cost
    END
  ) AS total_logistics_cost
FROM logistics_clean
GROUP BY order_id
) l
ON o.order_id = l.order_id

WHERE
  (o.quantity * o.selling_price) > 0
  AND
  (l.total_logistics_cost / (o.quantity * o.selling_price)) * 100 > 20

ORDER BY
  (l.total_logistics_cost / (o.quantity * o.selling_price)) * 100 DESC;

```

Task 9 — Output & Finding

OUTPUT SCREENSHOT

	order_id	order_value	total_logistics_cost	logistics_burden_percent
▶	13798	215	271	126.05%
	21716	220	277	125.91%
	14930	231	265	114.72%
	10135	263	299	113.69%
	14967	213	232	108.92%
	15173	305	312	102.3%
	9595	269	260	96.65%
	15910	218	209	95.87%
	4813	250	238	95.2%
	11884	354	324	91.53%
	13700	257	232	90.27%
	10193	329	296	89.97%
	21338	284	245	86.27%
	3348	416	353	84.86%
	20870	295	249	84.41%

RESULT

Several orders had logistics burden above 100%.

INSIGHT

Some low-value orders are expensive to fulfill compared with their order value.

ACTION

Review free shipping threshold should be kept.

Task 10 — Payment Fee Leakage

PROBLEM STATEMENT

Calculate gateway fees by payment method and evaluate their effect on profit.

SQL CONCEPTS USED

- SELECT
- COUNT
- ROUND
- SUM()
- CASE
- INNER JOIN
- LEFT JOIN
- Subquery
- WHERE
- GROUP BY
- ORDER BY

QUERY USED

```

1 • SELECT
2   o.payment_method,
3
4   COUNT(o.order_id) AS total_orders,
5
6   ROUND(
7     SUM(
8       CASE
9         WHEN
10          (
11            (o.quantity * o.selling_price)
12            -
13            CASE
14              WHEN d.total_discount IS NULL THEN 0
15              ELSE d.total_discount
16            END
17          ) <= 0
18          THEN 0
19          ELSE
20            (
21              (o.quantity * o.selling_price)
22              -
23              CASE
24                WHEN d.total_discount IS NULL THEN 0
25                ELSE d.total_discount
26              END
27            )
28          )
29          * pf.fee_percentage / 100
30      ),
31      2
32    ) AS total_payment_gateway_fee,
33
34   ROUND(
35     SUM(
36       (o.quantity * o.selling_price)
37       -
38       (o.quantity * p.cost_price)
39       -
40       CASE
41         WHEN d.total_discount IS NULL THEN 0
42         ELSE d.total_discount
43       END
44     )
45     -
46     CASE
47       WHEN
48         (
49           (o.quantity * o.selling_price)
50           -
51           CASE
52             WHEN d.total_discount IS NULL THEN 0
53             ELSE d.total_discount
54           END
55         )
56         * pf.fee_percentage / 100

```

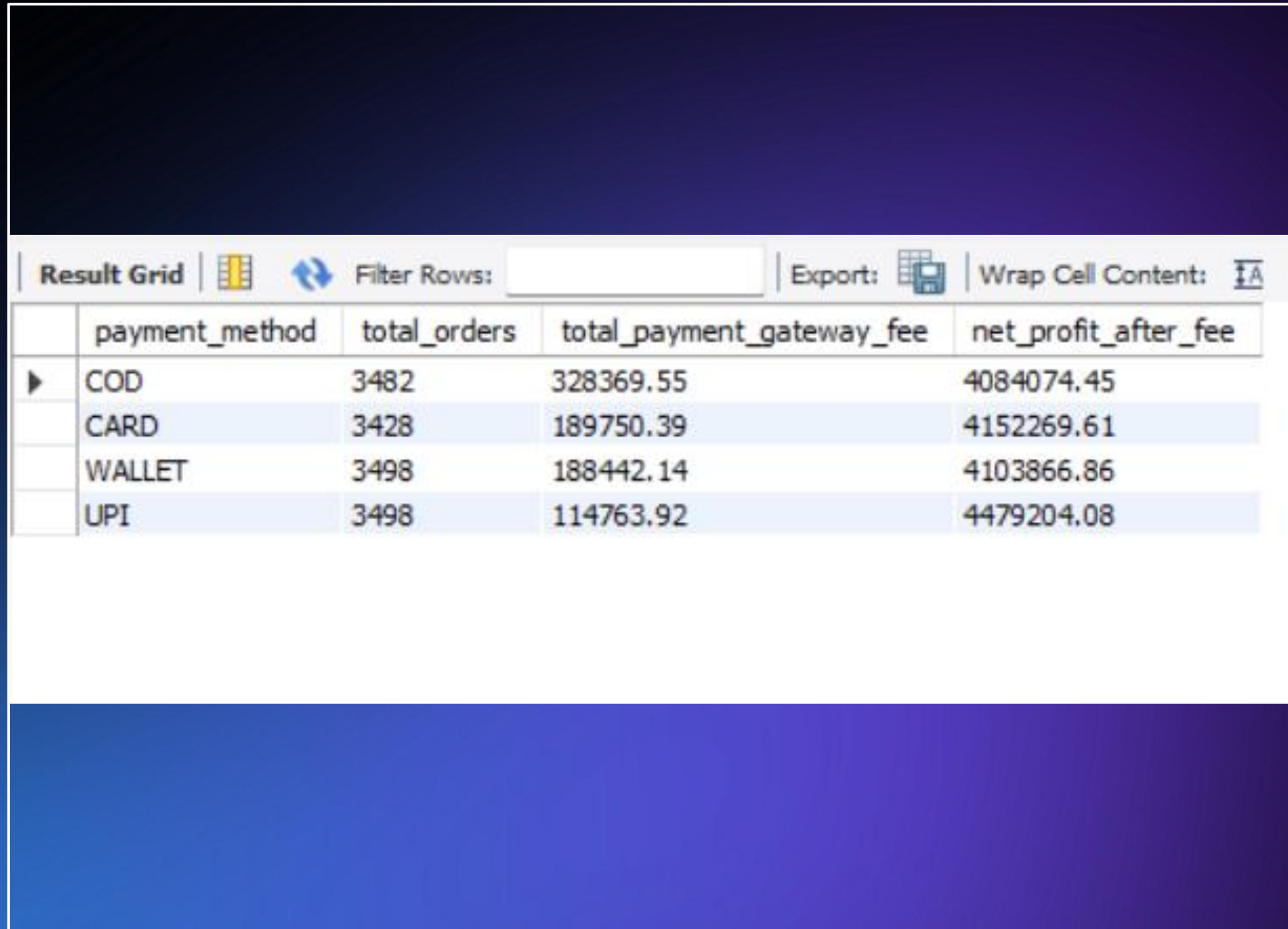
```

56         * pf.fee_percentage / 100
57       ) <= 0
58       THEN 0
59       ELSE
60         (
61           (o.quantity * o.selling_price)
62           -
63           CASE
64             WHEN d.total_discount IS NULL THEN 0
65             ELSE d.total_discount
66           END
67         )
68       )
69       * pf.fee_percentage / 100
70     )
71   ),
72   2
73 ) AS net_profit_after_fee
74
75 FROM orders_clean o
76
77 JOIN products_clean p
78   ON o.product_id = p.product_id
79
80 LEFT JOIN
81 (
82   SELECT
83     order_id,
84     SUM(discount_amount) AS total_discount
85   FROM discounts_clean
86   GROUP BY order_id
87 ) d
88   ON o.order_id = d.order_id
89
90 JOIN payment_fees_clean pf
91   ON o.payment_method = pf.payment_method
92
93 WHERE pf.fee_percentage IS NOT NULL
94
95 GROUP BY o.payment_method
96
97 ORDER BY total_payment_gateway_fee DESC;

```

Task 10 — Output & Finding

OUTPUT SCREENSHOT



The screenshot shows a data grid interface with a toolbar at the top containing 'Result Grid', a grid icon, a refresh icon, a 'Filter Rows:' input field, an 'Export:' button with a download icon, and a 'Wrap Cell Content:' toggle. Below the toolbar is a table with 5 columns: an expandable column, 'payment_method', 'total_orders', 'total_payment_gateway_fee', and 'net_profit_after_fee'. The table contains 4 rows of data for different payment methods: COD, CARD, WALLET, and UPI. The COD row is highlighted with a blue background.

	payment_method	total_orders	total_payment_gateway_fee	net_profit_after_fee
▶	COD	3482	328369.55	4084074.45
	CARD	3428	189750.39	4152269.61
	WALLET	3498	188442.14	4103866.86
	UPI	3498	114763.92	4479204.08

RESULT

COD fee was highest: ₹3,28,369.55. UPI was lowest: ₹1,14,763.92.

INSIGHT

COD creates the highest payment-fee burden among known methods.

ACTION

Review payment and COD fee has to be increased or reviewed.

Task 11A — Order-level Leakage Breakdown

PROBLEM STATEMENT

Calculate discount, return, logistics and payment-fee leakage for each order.

SQL CONCEPTS USED

- SELECT
- ROUND
- CASE
- Subqueries
- LEFT JOIN
- DISTINCT
- WHERE
- GROUP BY
- ORDER BY

QUERY USED

Section 1

```

1 • SELECT
2   x.order_id,
3
4   ROUND(x.discount_leakage, 2) AS discount_leakage,
5
6   ROUND(
7     CASE
8       WHEN x.is_returned = 1
9         AND x.after_discount_value > 0
10      THEN x.after_discount_value
11      ELSE 0
12    END,
13    2
14  ) AS return_leakage,
15
16  ROUND(x.logistics_leakage, 2) AS logistics_leakage,
17
18  ROUND(
19    CASE
20      WHEN x.fee_percentage IS NULL THEN NULL
21      WHEN x.after_discount_value <= 0 THEN 0
22      ELSE x.after_discount_value * x.fee_percentage / 100
23    END,
24    2
25  ) AS payment_fee_leakage,
26
27  ROUND(
28    x.discount_leakage
29
30    +
31
32    CASE
33      WHEN x.is_returned = 1
34        AND x.after_discount_value > 0
35      THEN x.after_discount_value
36      ELSE 0
37    END
38
39    +
40
41    x.logistics_leakage
42
43    +
44
45    CASE
46      WHEN x.fee_percentage IS NULL THEN 0
47      WHEN x.after_discount_value <= 0 THEN 0
48      ELSE x.after_discount_value * x.fee_percentage / 100
49    END,
50    2
51  ) AS total_measured_leakage
52

```

Section 2

```

53 FROM
54 (
55   SELECT
56     o.order_id,
57
58     CASE
59       WHEN d.total_discount IS NULL THEN 0
60       ELSE d.total_discount
61     END AS discount_leakage,
62
63     (o.quantity * o.selling_price)
64     -
65     CASE
66       WHEN d.total_discount IS NULL THEN 0
67       ELSE d.total_discount
68     END AS after_discount_value,
69
70     CASE
71       WHEN r.order_id IS NULL THEN 0
72       ELSE 1
73     END AS is_returned,
74
75     CASE
76       WHEN l.total_logistics_cost IS NULL THEN 0
77       ELSE l.total_logistics_cost
78     END AS logistics_leakage,
79
80     pf.fee_percentage
81   FROM orders_clean o
82
83   LEFT JOIN
84   (
85     SELECT
86       order_id,
87       SUM(d.discount_amount) AS total_discount
88     FROM discounts_clean
89     GROUP BY order_id
90   ) d
91   ON o.order_id = d.order_id
92
93   LEFT JOIN
94   (
95     SELECT DISTINCT
96       order_id
97     FROM returns_clean
98     WHERE return_flag = 'Y'
99   ) r
100   ON o.order_id = r.order_id
101
102   LEFT JOIN
103   (
104     SELECT
105       order_id,
106       SUM(
107         CASE
108           WHEN shipping_cost IS NULL THEN 0
109           ELSE shipping_cost
110         END
111         +
112         CASE
113           WHEN reverse_shipping_cost IS NULL THEN 0
114           ELSE reverse_shipping_cost
115         END
116       ) AS total_logistics_cost
117     FROM logistics_clean
118     GROUP BY order_id
119   ) l
120   ON o.order_id = l.order_id
121
122   LEFT JOIN payment_fees_clean pf
123   ON o.payment_method = pf.payment_method
124
125 ) x
126
127 ORDER BY total_measured_leakage DESC;
128
129
130
131

```

Task 11A — Output & Finding

OUTPUT SCREENSHOT

Result Grid  Filter Rows: Export:  Wrap Cell Content:  Fetch rows: 						
	order_id	discount_leakage	return_leakage	logistics_leakage	payment_fee_leakage	total_measured_leakage
▶	12670	12168	0	144	0	12312
	5622	12172	0	75	0	12247
	1308	12000	0	34	0	12034
	447	12000	0	0	0	12000
	911	0	8960	66	313.6	9339.6
	18065	0	8690	106	304.15	9100.15
	12406	0	8765	0	306.77	9071.78
	747	88	8462	210	296.17	9056.17
	16000	541	8344	0	166.88	9051.88
	9922	0	8760	0	183.96	8943.96
	7096	113	8532	0	298.62	8943.62
	3175	507	8248	0	164.96	8919.96
	17081	0	8800	0	105.6	8905.6
	8620	0	8770	0	105.24	8875.24
	20200	301	8274	0	289.59	8864.59

RESULT

Each order shows separate leakage columns and total measured leakage.

INSIGHT

Order-level leakage makes it easier to find the orders with the highest total loss.

TAKEAWAY

Use this detail before deciding where next query is needed.

Task 11B — Overall Leakage Summary

PROBLEM STATEMENT

Summarize all leakage types and identify the biggest leakage source overall.

SQL CONCEPTS USED

- WITH CTE
- SELECT
- CASE
- SUM
- ROUND
- INNER JOIN
- LEFT JOIN
- Subqueries
- UNION ALL
- ORDER BY
- NULL Handling

QUERY USED

Section 1

```
1 WITH order_leakage AS
2 (
3     SELECT
4         o.order_id,
5
6         -- Discount leakage
7         CASE
8             WHEN d.total_discount IS NULL THEN 0
9             ELSE d.total_discount
10        END AS discount_leakage,
11
12        -- Return leakage
13        CASE
14            WHEN r.order_id IS NOT NULL
15            AND
16            (
17                (o.quantity * o.selling_price)
18                -
19                CASE
20                    WHEN d.total_discount IS NULL THEN 0
21                    ELSE d.total_discount
22                END
23            ) > 0
24            THEN
25                (o.quantity * o.selling_price)
26                -
27                CASE
28                    WHEN d.total_discount IS NULL THEN 0
29                    ELSE d.total_discount
30                END
31            ELSE 0
32        END AS return_leakage,
33
34        -- Logistics leakage
35        CASE
36            WHEN l.total_logistics_cost IS NULL THEN 0
37            ELSE l.total_logistics_cost
38        END AS logistics_leakage,
39
40        -- Payment fee leakage
41        CASE
42            WHEN pf.fee_percentage IS NULL THEN 0
43
44            WHEN
45            (
46                (o.quantity * o.selling_price)
47                -
48                CASE
49                    WHEN d.total_discount IS NULL THEN 0
50                    ELSE d.total_discount
51                END
52            ) <= 0
53            THEN 0
54            ELSE
55            (
56                (o.quantity * o.selling_price)
57                -
58                CASE
59                    WHEN d.total_discount IS NULL THEN 0
60                    ELSE d.total_discount
61                END
62            )
63            * pf.fee_percentage / 100
64        END AS payment_fee_leakage
65
66 FROM orders_clean o
67
68 JOIN products_clean p
69     ON o.product_id = p.product_id
70
71 LEFT JOIN
72 (
73     SELECT
74         order_id,
75         SUM(discount_amount) AS total_discount
76     FROM discounts_clean
77     GROUP BY order_id
78 ) d
79     ON o.order_id = d.order_id
80
81 LEFT JOIN
82 (
83     SELECT DISTINCT
84         order_id
```

Section 2

```
84     SELECT DISTINCT
85         order_id
86     FROM returns_clean
87     WHERE return_flag = 'Y'
88 ) r
89     ON o.order_id = r.order_id
90
91 LEFT JOIN
92 (
93     SELECT
94         order_id,
95         SUM(
96             CASE
97                 WHEN shipping_cost IS NULL THEN 0
98                 ELSE shipping_cost
99             END
100         +
101         CASE
102             WHEN reverse_shipping_cost IS NULL THEN 0
103             ELSE reverse_shipping_cost
104         END
105     ) AS total_logistics_cost
106 FROM logistics_clean
107 GROUP BY order_id
108 ) l
109     ON o.order_id = l.order_id
110
111 LEFT JOIN payment_fees_clean pf
112     ON o.payment_method = pf.payment_method
113     ON o.payment_method = pf.payment_method
114
115 SELECT
116     'Discounts' AS leakage_type,
117     ROUND(SUM(discount_leakage), 2) AS total_leakage
118 FROM order_leakage
119
120 UNION ALL
121
122 SELECT
123     'Returns',
124     ROUND(SUM(return_leakage), 2)
125 FROM order_leakage
126
127 UNION ALL
128
129 SELECT
130     'Logistics',
131     ROUND(SUM(logistics_leakage), 2)
132 FROM order_leakage
133
134 UNION ALL
135
136 SELECT
137     'Payment Fees',
138     ROUND(SUM(payment_fee_leakage), 2)
139 FROM order_leakage
140
141 ORDER BY total_leakage DESC;
```

Task 11B — Output & Finding

OUTPUT SCREENSHOT

Result Grid  Filter Rows:		
	leakage_type	total_leakage
▶	Discounts	4583197
	Returns	2766054
	Logistics	940354
	Payment Fees	821326

RESULT

Discounts ₹45,83,197; Returns ₹27,66,054; Logistics ₹9,40,354; Payment Fees ₹8,21,326.

INSIGHT

Discounts are the largest measured leakage source, followed by returns.

ACTION

Discount has to be controlled, then return reduction.

Task 12 — Product Profit Ranking

PROBLEM STATEMENT

Rank all products by net profit contribution after measured leakage.

SQL CONCEPTS USED

- SELECT
- SUM
- ROUND
- CASE
- INNER JOIN
- LEFT JOIN
- Subqueries
- GROUP BY
- DENSE RANK
- ORDER BY

QUERY USED

Section 1

```
WITH order_profit AS
(
  SELECT
    o.product_id,
    (o.quantity * o.selling_price)
    - (o.quantity * p.cost_price)
    - CASE
      WHEN d.total_discount IS NULL THEN 0
      ELSE d.total_discount
    END
    - CASE
      WHEN r.order_id IS NOT NULL
      AND
      (
        (o.quantity * o.selling_price)
        -
        CASE
          WHEN d.total_discount IS NULL THEN 0
          ELSE d.total_discount
        ) > 0
      ) THEN
      (o.quantity * o.selling_price)
      -
      CASE
        WHEN d.total_discount IS NULL THEN 0
        ELSE d.total_discount
      END
    ELSE 0
    END
    - CASE
      WHEN l.total_logistics_cost IS NULL THEN 0
      ELSE l.total_logistics_cost
    END
    - CASE
      WHEN pf.fee_percentage IS NULL THEN 0
      WHEN
      (
        (o.quantity * o.selling_price)
        -
        CASE
          WHEN d.total_discount IS NULL THEN 0
          ELSE d.total_discount
        ) <= 0
      ) THEN 0
      ELSE
      (
        (o.quantity * o.selling_price)
        -
        CASE
          WHEN d.total_discount IS NULL THEN 0
          ELSE d.total_discount
        )
        * pf.fee_percentage / 100
      )
    END
  AS net_profit
  FROM orders_clean o
  JOIN products_clean p
    ON o.product_id = p.product_id
  LEFT JOIN
  (
    SELECT
      order_id,
      SUM(discount_amount) AS total_discount
    FROM discounts_clean
    GROUP BY order_id
  ) d
    ON o.order_id = d.order_id
  LEFT JOIN
  (
    SELECT DISTINCT order_id
    FROM returns_clean
    WHERE return_flag = 'Y'
  ) r
    ON o.order_id = r.order_id
  LEFT JOIN
  (
    SELECT
      order_id,
      SUM(
        CASE
          WHEN shipping_cost IS NULL THEN 0
          ELSE shipping_cost
        END
        +
        CASE
          WHEN reverse_shipping_cost IS NULL THEN 0
          ELSE reverse_shipping_cost
        END
      ) AS total_logistics_cost
    FROM logistics_clean
    GROUP BY order_id
  ) l
    ON o.order_id = l.order_id
  LEFT JOIN payment_fees_clean pf
    ON o.payment_method = pf.payment_method
),
product_profit AS
(
  SELECT
    product_id,
    SUM(net_profit) AS total_net_profit
  FROM order_profit
  GROUP BY product_id
)
SELECT
  product_id,
  ROUND(total_net_profit, 2) AS total_net_profit,
  DENSE_RANK() OVER
  (
    ORDER BY total_net_profit DESC
  ) AS profit_rank
FROM product_profit
ORDER BY profit_rank;
```

Section 2

```
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Task 12 — Output & Finding

OUTPUT SCREENSHOT

	product_id	total_net_profit	profit_rank
▶	618	58467.05	1
	124	58035.02	2
	963	56535.8	3
	960	54603.71	4
	1640	54051.21	5
	280	52744.31	6
	176	48198.6	7
	1568	47932.99	8
	973	47044.19	9
	741	46330.18	10
	1219	46004.61	11
	887	45707.47	12
	663	45687.88	13
	1743	45450.05	14
	1198	44302.01	15
	167	42841.71	16

RESULT

Product 618 ranked #1 with net profit ₹58,467.05.

INSIGHT

The ranking shows products that contribute most after leakage, not just high sales.

ACTION

High-profit products should be prioritised and investigate low-ranked products.

Task 13 — Category Margin Stability

PROBLEM STATEMENT

Identify categories with high variation in profit margins across orders.

SQL CONCEPTS USED

- SELECT
- COUNT
- ROUND
- AVG()
- STDDEV
- MIN
- MAX
- INNER JOIN
- WHERE
- GROUP BY
- ORDER BY

QUERY USED

Section 1

```
1 • WITH order_margin AS
2 (
3     SELECT
4         p.category,
5
6         (
7             (
8                 (o.quantity * o.selling_price)
9                 -
10                (o.quantity * p.cost_price)
11            )
12            /
13            (o.quantity * o.selling_price)
14        ) * 100 AS profit_margin_percent
15
16     FROM orders_clean o
17
18     JOIN products_clean p
19         ON o.product_id = p.product_id
20
21     WHERE
22         (o.quantity * o.selling_price) > 0
23 )
24
25 SELECT
26     category,
27
28     COUNT(*) AS total_orders,
```

Section 2

```
1     ROUND(
2         AVG(profit_margin_percent),
3         2
4     ) AS avg_profit_margin,
5
6     ROUND(
7         STDDEV(profit_margin_percent),
8         2
9     ) AS margin_stddev,
10
11    ROUND(
12        MIN(profit_margin_percent),
13        2
14    ) AS minimum_margin,
15
16    ROUND(
17        MAX(profit_margin_percent),
18        2
19    ) AS maximum_margin
20
21 FROM order_margin
22
23 GROUP BY category
24
25 ORDER BY margin_stddev DESC;
```

Task 13 — Output & Finding

OUTPUT SCREENSHOT

	category	total_orders	avg_profit_margin	margin_stddev	minimum_margin	maximum_margin
►	Fashion	3018	35.28	59.53	-324.15	95.38
	Unknown	2297	38.99	58.81	-298.12	95.32
	Beauty	2664	36.23	58.73	-297.73	95.31
	Electronics	3311	37.82	56.82	-323.04	95.15
	Grocery	2552	39.33	56.53	-319.9	95.54
	Home	2756	40.37	55.48	-324.64	95.21

RESULT

Fashion had the highest margin variation: STDDEV 59.53.

INSIGHT

Fashion margins were the least consistent across orders in this dataset.

ACTION

Review pricing and cost consistency in high-variation categories.

Task 14 — High-Risk Customers

PROBLEM STATEMENT

Identify customers whose return rate is higher than the marketplace average return rate.

SQL CONCEPTS USED

- SELECT
- DISTINCT
- COUNT
- SUM
- CASE
- LEFT JOIN
- CROSS JOIN
- WHERE
- GROUP BY
- ORDER BY

QUERY USED

Section 1

```
WITH returned_orders AS
(
    SELECT DISTINCT
        order_id
    FROM returns_clean
    WHERE return_flag = 'Y'
),
customer_returns AS
(
    SELECT
        o.customer_id,
        COUNT(o.order_id) AS total_orders,
        SUM(
            CASE
                WHEN r.order_id IS NULL THEN 0
                ELSE 1
            END
        ) AS returned_orders,
        (
            SUM(
                CASE
                    WHEN r.order_id IS NULL THEN 0
                    ELSE 1
                END
            )
        ) / COUNT(o.order_id)
        ) * 100 AS customer_return_rate
    FROM orders_clean o
    LEFT JOIN returned_orders r
        ON o.order_id = r.order_id
    GROUP BY o.customer_id
),
marketplace_rate AS
(
    SELECT
        (
            SUM(
                CASE
                    WHEN r.order_id IS NULL THEN 0
                    ELSE 1
                END
            )
        ) / COUNT(o.order_id)
        ) * 100 AS marketplace_return_rate
    FROM orders_clean o
```

Section 2

```
LEFT JOIN returned_orders r
    ON o.order_id = r.order_id
)

SELECT
    c.customer_id,
    c.total_orders,
    c.returned_orders,
    ROUND(
        c.customer_return_rate,
        2
    ) AS customer_return_rate,
    ROUND(
        m.marketplace_return_rate,
        2
    ) AS marketplace_return_rate,
    ROUND(
        c.customer_return_rate
        - m.marketplace_return_rate,
        2
    ) AS above_marketplace_by
FROM customer_returns c
CROSS JOIN marketplace_rate m

WHERE
    c.customer_return_rate
    > m.marketplace_return_rate

ORDER BY
    above_marketplace_by DESC,
    c.total_orders DESC;
```

Task 14 — Output & Finding

OUTPUT SCREENSHOT

Result Grid						
Filter Rows:		Export:		Wrap Cell Content:		
	customer_id	total_orders	returned_orders	customer_return_rate	marketplace_return_rate	above_marketplace_by
▶	3998	2	2	100.00	5.88	94.12
	1315	2	2	100.00	5.88	94.12
	2865	2	2	100.00	5.88	94.12
	8907	2	2	100.00	5.88	94.12
	6362	2	2	100.00	5.88	94.12
	2893	2	2	100.00	5.88	94.12
	5571	2	2	100.00	5.88	94.12
	6800	1	1	100.00	5.88	94.12
	7386	1	1	100.00	5.88	94.12
	5508	1	1	100.00	5.88	94.12
	7543	1	1	100.00	5.88	94.12
	6008	1	1	100.00	5.88	94.12
	7981	1	1	100.00	5.88	94.12
	1468	1	1	100.00	5.88	94.12
	2236	1	1	100.00	5.88	94.12

RESULT

Marketplace return rate was 5.88%; several customers showed 100% return rate.

INSIGHT

Customers above the benchmark should be reviewed, but small order counts need to be seen.

ACTION

Use return rate together with total order count before action.

Task 15 — Executive Profitability Summary

PROBLEM STATEMENT

Create one consolidated summary of sales, costs, leakage and final net profit.

SQL CONCEPTS USED

- SELECT
- SUM
- ROUND
- CASE
- INNER JOIN
- LEFT JOIN
- Subqueries
- Arithmetic Calculations

QUERY USED

Section 1

```
WITH order_financials AS
(
  SELECT
    o.order_id,

    -- Sales
    (o.quantity * o.selling_price) AS sales,

    -- Product Cost
    (o.quantity * p.cost_price) AS product_cost,

    -- Discount
    CASE
      WHEN d.total_discount IS NULL THEN 0
      ELSE d.total_discount
    END AS discount_amount,

    -- Return Loss
    CASE
      WHEN r.order_id IS NOT NULL
      AND
      (
        (o.quantity * o.selling_price)
        -
        CASE
          WHEN d.total_discount IS NULL THEN 0
          ELSE d.total_discount
        END
      ) > 0
    THEN
      (o.quantity * o.selling_price)
      -
      CASE
        WHEN d.total_discount IS NULL THEN 0
        ELSE d.total_discount
      END
    ELSE 0
    END AS return_loss,

    -- Logistics Cost
    CASE
      WHEN l.total_logistics_cost IS NULL THEN 0
      ELSE l.total_logistics_cost
    END AS logistics_cost,

    -- Payment Fee
    CASE
      WHEN pf.fee_percentage IS NULL THEN 0
      WHEN
      (
        (o.quantity * o.selling_price)
        -
        CASE
          WHEN d.total_discount IS NULL THEN 0
          ELSE d.total_discount
        END
      ) <= 0
      ) <= 0
    THEN 0
    ELSE
      (
        (o.quantity * o.selling_price)
        -
        CASE
          WHEN d.total_discount IS NULL THEN 0
          ELSE d.total_discount
        END
      )
      * pf.fee_percentage / 100
    END AS payment_fee

  FROM orders_clean o

  JOIN products_clean p
    ON o.product_id = p.product_id

  LEFT JOIN
  (
    SELECT
      order_id,
      SUM(discount_amount) AS total_discount
    FROM discounts_clean
    GROUP BY order_id
  ) d
    ON o.order_id = d.order_id
```

Section 2

```
LEFT JOIN
(
  SELECT DISTINCT
    order_id
  FROM returns_clean
  WHERE return_flag = 'Y'
) r
  ON o.order_id = r.order_id

LEFT JOIN
(
  SELECT
    order_id,
    SUM(
      CASE
        WHEN shipping_cost IS NULL THEN 0
        ELSE shipping_cost
      END
      +
      CASE
        WHEN reverse_shipping_cost IS NULL THEN 0
        ELSE reverse_shipping_cost
      END
    ) AS total_logistics_cost
  FROM logistics_clean
  GROUP BY order_id
) l
  ON o.order_id = l.order_id

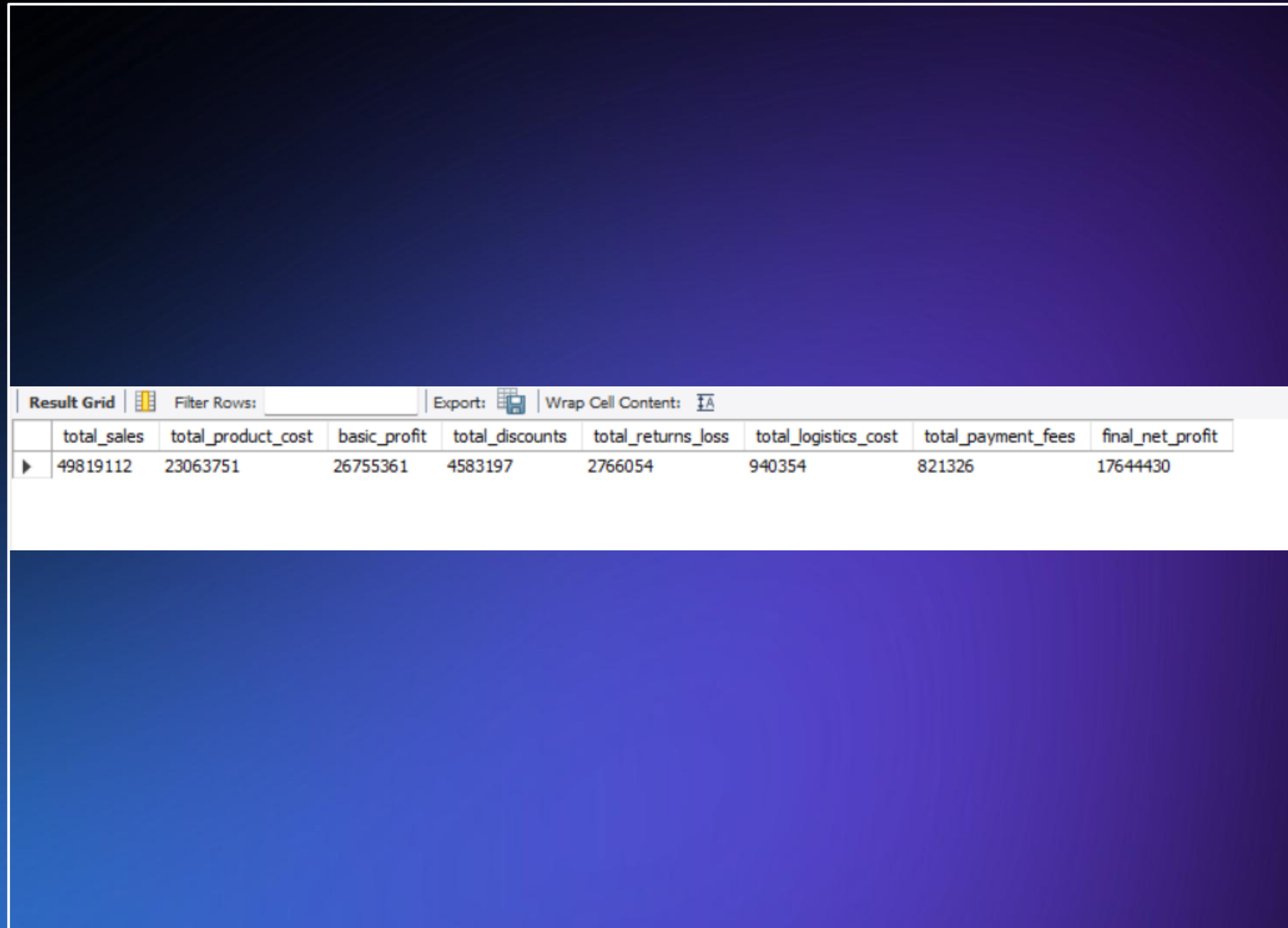
  ON o.payment_method = pf.payment_method
)

SELECT
  ROUND(SUM(sales), 2) AS total_sales,
  ROUND(SUM(product_cost), 2) AS total_product_cost,
  ROUND(
    SUM(sales - product_cost),
    2
  ) AS basic_profit,
  ROUND(
    SUM(discount_amount),
    2
  ) AS total_discounts,
  ROUND(
    SUM(return_loss),
    2
  ) AS total_returns_loss,
  ROUND(
    SUM(logistics_cost),
    2
  ) AS total_logistics_cost,
  ROUND(
    SUM(payment_fee),
    2
  ) AS total_payment_fees,
  ROUND(
    SUM(
      (sales - product_cost)
      - discount_amount
      - return_loss
      - logistics_cost
      - payment_fee
    ),
    2
  ) AS final_net_profit

FROM order_financials;
```

Task 15 — Output & Finding

OUTPUT SCREENSHOT



The screenshot shows a data grid interface with a toolbar at the top containing 'Result Grid', a grid icon, 'Filter Rows:', an 'Export:' button with a download icon, and a 'Wrap Cell Content:' button with a text icon. Below the toolbar is a table with 9 columns: an empty header cell, 'total_sales', 'total_product_cost', 'basic_profit', 'total_discounts', 'total_returns_loss', 'total_logistics_cost', 'total_payment_fees', and 'final_net_profit'. The first data row contains the values: 49819112, 23063751, 26755361, 4583197, 2766054, 940354, 821326, and 17644430.

	total_sales	total_product_cost	basic_profit	total_discounts	total_returns_loss	total_logistics_cost	total_payment_fees	final_net_profit
▶	49819112	23063751	26755361	4583197	2766054	940354	821326	17644430

RESULT

Sales ₹4.98 Cr; basic profit ₹2.68 Cr; final measured net profit ₹1.76 Cr.

INSIGHT

Discounts, returns, logistics and payment fees reduced the basic profit significantly.

ACTION

Focus first on discount governance and preventable return reduction.

Final Business Findings & Recommendations

₹4.98 Cr	₹2.68 Cr	₹91.11 L	₹1.76 Cr
Total Sales	Basic Profit	Measured Leakage	Final Net Profit

LARGEST MEASURED LEAKAGE SOURCES

- Discounts — ₹45.83 L
- Returns — ₹27.66 L
- Logistics — ₹9.40 L
- Payment Fees — ₹8.21 L

BUSINESS FINDINGS

- Discounts were the largest measured leakage source.
- Returns were the second-largest leakage source.
- Discounted orders had lower average profit per order.
- COD created the highest known payment-fee burden.
- Some orders had logistics cost greater than order value.

RECOMMENDED FOCUS

1. Control discount depth and measure discount effectiveness.
2. Reduce preventable returns, especially damage-related returns.
3. Review high-logistics-burden orders and COD fee increase.
4. Use product profitability and customer return-rate analysis for future decisions.

Thank You

D. Jaya Kumar